

วันที่ 3

Global Constraints

— เครื่องมือสำคัญ

Constraint สำเร็จรูปที่ Solver มี algorithm พิเศษ — แก้ได้เร็วกว่าเขียนเอง

alldifferent

Assignment

cumulative

Scheduling

disjunctive

No-overlap

table

Lookup

กำหนดการวันที่ 3

09:00–09:45

**alldifferent***Assignment, no-duplicate — ใช้กับอะไรบ้าง*

09:45–11:00

**cumulative***Resource-constrained scheduling — คน, เครื่องจักร, เวลา*

11:00–12:00

**Workshop 3A***Maintenance Scheduling ด้วย cumulative*

13:00–14:00

**table constraint***เลือกจากชุดค่าที่กำหนด — material, spec, ราคา*

14:00–15:00

**disjunctive + Job Shop***No-overlap, precedence — ตารางการผลิต*

15:00–16:00

**Workshop 3B***Job Shop Scheduling — 4 ออร์เดอร์ 3 สถานี*

ทำไมต้องใช้ Global Constraints?

Global Constraints = constraint สำเร็จรูปที่ผู้เชี่ยวชาญออกแบบมาพร้อม algorithm พิเศษ

✗ เขียนเอง (ช้า)

```
% n=10, ต้องต่างกันทุกคู่  
  
constraint forall(i,j in 1..10  
    where i<j)(x[i] != x[j]);  
  
% = 45 constraints!  
  
% solver ต้อง check ทุกคู่
```

✓ Global Constraint (เร็ว)

```
include "globals.mzn";  
  
constraint alldifferent(x);  
  
% 1 constraint, solver ใช้  
% arc-consistency algorithm
```



Global constraints ทำให้ solve ได้เร็วกว่า และโค้ดสั้นกว่าอย่างมาก

alldifferent — ทุกค่าต้องต่างกัน

$\text{alldifferent}([x_1, x_2, \dots, x_n]) \rightarrow x_1 \neq x_2, x_1 \neq x_3, \dots, x_{n-1} \neq x_n$ (ทุกคู่)

```
include "globals.mzn"; % หรือ globals.mzn สำหรับทุก global

% มอบหมาย task ให้ worker — แต่ละ worker ได้ task ต่างกัน

array[WORKERS] of var TASKS: assigned;

constraint alldifferent(assigned);
```

ใช้กับปัญหาไหนบ้าง:



Assignment problems

มอบหมายงาน, จัดกะ —
ห้ามซ้ำ



Machine assignment

ออเดอร์แต่ละชิ้นใช้เครื่อง
ต่างกัน



Sudoku / N-Queens

แต่ละแถว/คอลัมน์ต้องมีค่า
1-n ครบและไม่ซ้ำ



Permutations

กำหนดให้เป็นการ
เรียงลำดับ

ตัวอย่าง: มอบหมายงาน Minimize เวลา

```
include "alldifferent.mzn";  
  
enum WORKERS = { Alice, Bob, Carol, Dave };  
enum TASKS   = { TaskA, TaskB, TaskC, TaskD };  
  
array[WORKERS,TASKS] of int: dur;  
array[WORKERS] of var TASKS: assigned;  
  
constraint alldifferent(assigned);  
  
solve minimize  
  sum(w in WORKERS)(dur[w, assigned[w]]);
```

◀ ตัวแปรแต่ละตัว
เป็น enum TASKS

◀ alldifferent!
ทุกคนได้งานต่างกัน

◀ minimize เวลารวม
ทุกคน

cumulative — Resource-Constrained Scheduling

```
cumulative(start, duration, resource_req, capacity)
```

start[]

→ เวลาเริ่มต้นของแต่ละงาน (decision variables)

duration[]

→ ระยะเวลาของแต่ละงาน (ค่าคงที่หรือ variable)

resource_req[]

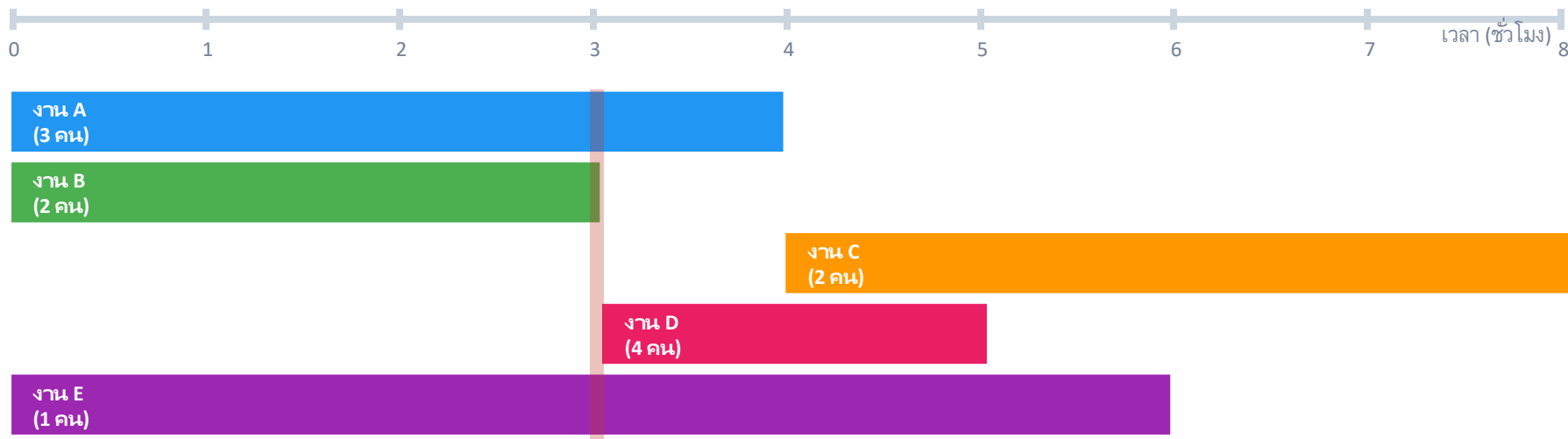
→ ทรัพยากรที่งานนั้นต้องการในแต่ละช่วง

capacity

→ ทรัพยากรทั้งหมดที่มี (ต้องไม่เกินในทุกเวลา)

```
include "cumulative.mzn"; constraint cumulative(start, duration, workers_needed, total_workers);
```

cumulative — ภาพประกอบ



ถ้าเวลา $t=3$: ทำงาน $A(3)+B(2)+E(1) = 6$ คน เกิน $capacity=5$ — cumulative จะบังคับให้ shift งานใหม่

ตัวอย่าง: Maintenance Scheduling

```
include "cumulative.mzn";

enum JOBS; array[JOBS] of int: dur, workers;

int: total_workers, deadline;

array[JOBS] of var 0..deadline: start;

var 0..deadline: end_time;

% cumulative: ทุกเวลา ช่วงรวม <= total_workers

constraint cumulative(start, dur, workers, total_workers);

% forall job: เสร็จก่อน end_time

constraint forall(j in JOBS)(start[j]+dur[j] <= end_time);

solve minimize end_time;
```

◀ ตัวแปรสำคัญ:
เวลาเริ่มของแต่ละงาน

◀ 1 constraint ครอบคลุมเวลา
— ไม่ต้องวน loop

◀ minimize เวลาเสร็จรวม
(makespan)



Workshop 3A — Maintenance Scheduling

เวลา: 60 นาที | ไฟล์: workshop3a_starter.mzn

โจทย์: งานซ่อมบำรุง 6 งาน ช่าง 5 คน ต้องเสร็จใน 12 ชม.

A(4ชม.,3คน), B(3ชม.,2คน), C(5ชม.,2คน), D(2ชม.,4คน), E(6ชม.,1คน), F(3ชม.,3คน)

งาน C ต้องทำหลังงาน A เสร็จ

1 เพิ่ม cumulative constraint:

2 เพิ่ม precedence: A ก่อน C:

3 ทุกงานเสร็จก่อน end_time:

4 solve minimize end_time:

5 (ท้าทาย) เพิ่ม D หลัง B:

ช่วงบ่าย

table constraint และ Job Shop Scheduling

- `table(x, allowed_tuples)`

- disjunctive + precedence

- minimize makespan

table — เลือกจากชุดค่าที่กำหนดไว้

`table(x, allowed_tuples)` → `x` ต้องเป็น row ใด row หนึ่งในตาราง

```
include "table.mzn";

% ตาราง: [material_id, hardness, weight, cost]
array[1..5,1..4] of int: mat_table = [
    1, 90, 800, 50 | % Steel
    2, 60, 270, 80 | % Aluminum ...  ];

var int: mat_id; var int: hardness; ...

constraint table([mat_id, hardness, weight, cost], mat_table);
```

ใช้กับปัญหาอะไร:

ตาราง spec/ราคาสินค้า

เลือกวัสดุให้ตรงสเปค, ราคาต่ำสุด

ตารางกะพนักงาน

ชุดกะที่อนุญาต เช่น (เช้า,บ่าย) หรือ (บ่าย,ดึก) ห้าม (ดึก,เช้า)

Bill of Materials

ชุดส่วนผสมที่เป็นไปได้สำหรับแต่ละผลิตภัณฑ์

disjunctive — ห้ามทำพร้อมกัน

`disjunctive(start[], duration[])` → ทุก task ต้องไม่ overlap กัน

เขียนเอง vs. disjunctive:

% ❌ เขียนเอง ($O(n^2)$ constraints)

```
constraint forall(i,j in JOBS
  where i < j)(s[i]+d[i]<=s[j]
  \ / s[j]+d[j]<=s[i]);
```

% ✅ disjunctive (1 constraint)

```
include "disjunctive.mzn";

constraint disjunctive
  (start, duration);
```

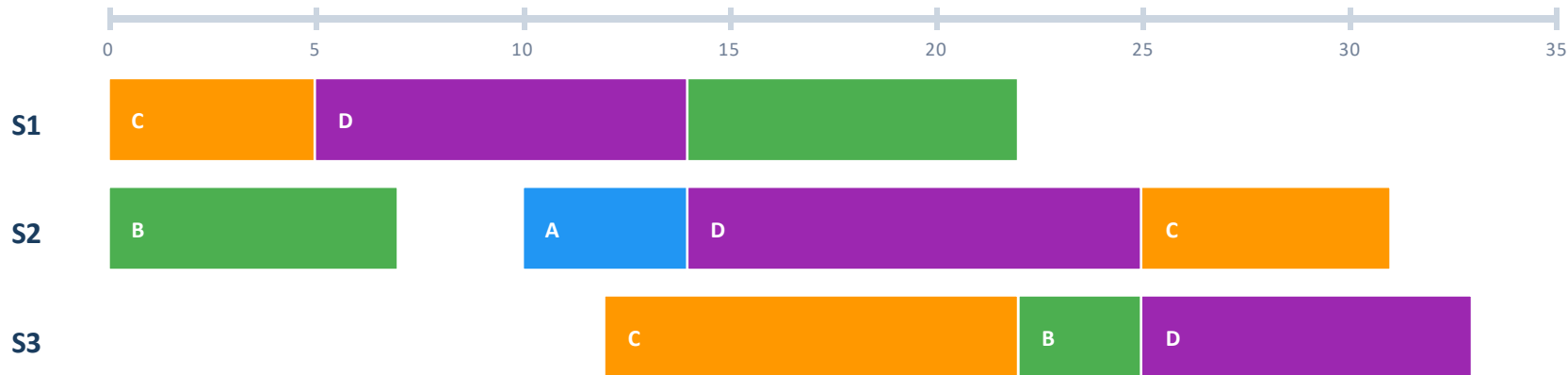
ใน Job Shop: disjunctive ต่อเครื่องจักร

% ทุก task ที่ใช้เครื่องเดียวกันต้องไม่ overlap

```
constraint forall(m in MACHINES)(
  disjunctive([s[j,m] | j in JOBS], [d[j,m] | j in JOBS]));
```

Job Shop Scheduling — แนวคิด

Job = ออร์เดอร์ผลิต | Task = ขั้นตอนในแต่ละสถานี | Makespan = เวลาที่ทุก job เสร็จ



กฎ: แต่ละ Job ผ่าน S1→S2→S3 ตามลำดับ | แต่ละสถานีทำ Job ได้ทีละ 1 | Minimize makespan

Job Shop — โมเดล MiniZinc

```
include "disjunctive.mzn";

enum JOB; enum TASK;

array[JOB,TASK] of int: d; TASK: t_last = max(TASK);

int: total = sum(j,t)(d[j,t]);

array[JOB,TASK] of var 0..total: s;
var 0..total: makespan;

% Precedence: S1→S2→S3
constraint forall(j in JOB)(
  forall(t in TASK where t<t_last)
    (s[j,t]+d[j,t]<=s[j,enum_next(TASK,t)]));

% No-overlap: ทุกสถานีทีละ 1 job
constraint forall(t in TASK)(
  disjunctive([s[j,t]|j in JOB],[d[j,t]|j in JOB]));

solve minimize makespan;
```

◀ เวลาเริ่มของทุก
task ของทุก job

◀ Precedence
S→S→S ตามลำดับ

◀ disjunctive ต่อ station
— 1 constraint ต่อสถานี

Job Shop — Data File และผลลัพธ์

 jobshop_data.dzn

```
JOB = J(1..3);
TASK = T(1..3);

% d[job, task] = เวลา (นาที)

d = [| 10, 8, 12    % Job 1
      | 7, 15, 5    % Job 2
      | 12, 6, 10 |]; % Job 3
```

 ผลลัพธ์

Makespan = 30 นาที

Job	S1	S2	S3
1	5	9	17
2	0	10	25
3	0	7	13

สิ่งที่น่าสังเกต:

<code>enum_next(TASK, t)</code>	เป็น built-in ที่ดึง element ถัดไปใน enum — ไม่ต้อง hardcode t+1
<code>disjunctive</code> ต่อ TASK	ใช้ list comprehension <code>[s[j,t] j in JOB]</code> สร้าง array แบบ on-the-fly
<code>makespan = last task</code>	เป็น variable แยกต่างหาก — bound คือ total duration ทั้งหมด



Workshop 3B — Job Shop Scheduling

เวลา: 60 นาที | ไฟล์: workshop3b_starter.mzn

โจทย์: 4 ออร์เดอร์ (A,B,C,D) ผ่าน 3 สถานี ($S1 \rightarrow S2 \rightarrow S3$) เสมอ

$A=[10,8,12]$ $B=[7,15,5]$ $C=[12,6,10]$ $D=[9,11,8]$ (นาที)

แต่ละสถานีทำได้ทีละ 1 ออร์เดอร์ — Minimize makespan

- 1 เพิ่ม Precedence:
- 2 เพิ่ม Disjunctive:
- 3 เพิ่ม makespan:
- 4 อ่านผล + Gantt: วาด Gantt chart บนกระดาษจากค่า start times ที่ได้
- 5 (ท้าทาย) Constraint เพิ่ม: D ต้องเริ่ม S1 ไม่ก่อนหน้าที่ 5 → makespan เปลี่ยนไปไหม?

สรุปวันที่ 3 — สิ่งที่ต้องจำ



alldifferent — ไม่ต้องเขียน $O(n^2)$ constraints

ใช้กับ assignment, permutation, Sudoku — 1 constraint แทน $n*(n-1)/2$ constraints



cumulative — ทรัพยากรต้องไม่เกิน capacity ทุกเวลา

`cumulative(start, dur, resource_req, capacity)` — หัวใจของ resource scheduling



disjunctive — ห้าม overlap บนเครื่องจักรเดียวกัน

Job Shop = precedence (ลำดับใน job) + disjunctive (ห้ามใช้เครื่องพร้อมกัน)



table — เลือกจาก tuple ที่อนุญาต

ใช้กับ spec สินค้า, ตาราง config, ชุดค่าที่เป็นไปได้ตามกฎ

Quick Reference: Global Constraints

```
include "globals.mzn"; % includes everything

% — ALLDIFFERENT —
alldifferent(array[int] of var int: x)

% — CUMULATIVE —
cumulative(array of var int: s, % start times
           array of var int: d, % durations
           array of var int: r, % resource req
           var int: b)          % capacity

% — DISJUNCTIVE —
disjunctive(array of var int: s, array of int: d)

% — TABLE —
table(array[int] of var int: x, array[int,int] of int: t)
```

Quick Reference: Scheduling Patterns

% — PRECEDENCE (A ต้องเสร็จก่อน B) —————

```
constraint start_A + duration_A <= start_B;
```

% — JOB SHOP STRUCTURE —————

% 1. Precedence ตามลำดับ task ใน job เดียวกัน

```
constraint forall(j in JOB, t in TASK where t < t_last)(
  s[j,t] + d[j,t] <= s[j, enum_next(TASK,t)]);
```

% 2. Disjunctive: ทุก job ที่ใช้สถานีเดียวกันต้องไม่ overlap

```
constraint forall(t in TASK)(
  disjunctive([s[j,t] | j in JOB], [d[j,t] | j in JOB]));
```

% 3. Makespan

```
constraint forall(j in JOB)(s[j,t_last] + d[j,t_last] <= makespan);
```

```
solve minimize makespan;
```

ข้อผิดพลาดที่พบบ่อย

✗ `cumulative(start, dur, req, cap) % ลืม include`

✓ `include "cumulative.mzn"; % ต้องมีก่อนทุกครั้ง`

ลืมนำ `include` ทำให้ `error: undefined identifier`

✗ `s[j,t] + d[j,t] <= s[j, t+1] % t เป็น enum`

✓ `s[j,t] + d[j,t] <= s[j, enum_next(TASK,t)]`

ถ้า `TASK` เป็น `enum` ต้องใช้ `enum_next` ไม่ใช่ `t+1`

✗ `disjunctive(start_1, dur_1, start_2, dur_2)`

✓ `disjunctive([s[j,t]|j in JOBS], [d[j,t]|j in JOBS])`

`disjunctive` รับ array ของ start times ทุก job พร้อมกัน — ไม่ใช่ทีละคู่

✗ `solve satisfy % แต่อยากได้ makespan น้อยสุด`

✓ `solve minimize makespan; % ต้องระบุ objective`

`satisfy` หาแค่คำตอบแรก — ไม่การันตีว่า `optimal`

เลือก Global Constraint ให้ถูกต้อง

ต้องการ	ใช้	ตัวอย่าง
ต้องการให้ค่าไม่ซ้ำกัน	<code>alldifferent(x)</code>	Assignment, Sudoku, Permutation
ทรัพยากรต้องไม่เกิน capacity	<code>cumulative(s,d,r,cap)</code>	คน, เครื่องจักร, งบ
เครื่องจักรทำทีละ 1	<code>disjunctive(s,d)</code>	Job Shop, Machine Scheduling
เลือกจาก spec ที่กำหนด	<code>table(x, tuples)</code>	วัสดุ, กะ, Bill of Materials
ลำดับใน job	<code>precedence constraint</code>	$s[j,t] + d[j,t] \leq s[j,t+1]$
ทุก job เสร็จ $\rightarrow$ makespan	<code>forall(j)(s[j,last]+d <= ms)</code>	Minimize total time

Day 4 Preview

Scheduling เเซิงลิก และ Employee Rostering

Job Shop พร้อม predicate — encapsulate no_overlap

Flexible Job Shop — เลือกเครื่องจักรได้

Employee Rostering — 3 กะ, demand, fairness

regular constraint — กฎ shift pattern ซ้ำซ้อน